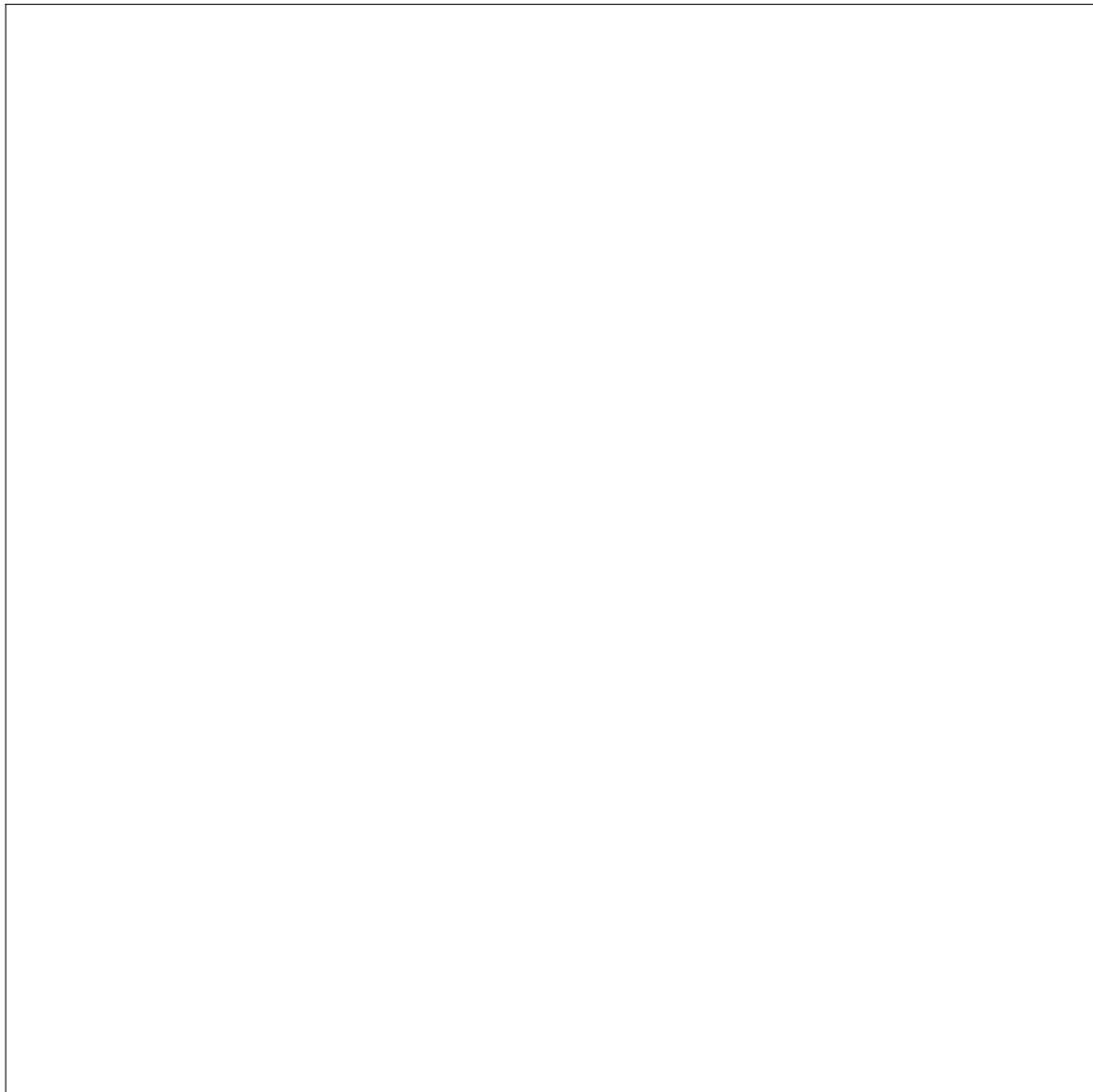


Содержание

1	Наводящие размышления	2
2	Повторные выборки (Ресемплинг)	3
2.1	Введение	3
2.2	Джекнайф	6
2.2.1	Немного о самом методе	6
2.2.2	Джекнайф-оценка смещения	10
2.2.3	Джекнайф-оценка дисперсии	15
2.3	Бутстрэп	16
2.3.1	Общая идея оценок методом бутстрэп	16
2.3.2	Метод бутстрэп	17
2.3.3	Построение доверительных интервалов	20
2.3.4	Некоторые финальные замечания	21
3	Ансамбли моделей	21
3.1	Общее понятие ансамбля	21
3.2	Бэггинг	26
3.2.1	Пример решения задачи классификации	26
3.2.2	Пример задачи регрессии	29
3.3	Адаптивный бустинг для двухклассовой классификации (AdaBoost)	33
3.3.1	Пример	36
3.4	Стекинг (Stacking)	41
3.4.1	Пример на пальцах	42
3.5	Случайный лес	44
4	Многоклассовая классификация	46
4.1	Один против всех (one-vs-all)	47
4.1.1	Пример	48
4.2	Все против всех (all-vs-all)	51
4.2.1	Пример	53



3.3 Адаптивный бустинг для двухклассовой классификации (AdaBoost)

Перейдем к рассмотрению еще одного метода построения ансамблей – бустингу. Основное отличие этого метода от ранее рассмотренного бэггинга заключается в том, что слабые ученики (базовые алгоритмы) обучаются теперь не независимо, а последовательно, учитывая опыт предыдущего. Так, каждый последующий ученик начинает придавать большее значение тем наблюдениям из тренировочного набора данных, на которых ошиблись предыдущие ученики.

Рассмотрим алгоритм адаптивного бустинга, причем применительно только к задаче двухклассовой классификации. Итак, предположим, что $Y = \{-1, 1\}$, $x_1, x_2, \dots, x_n$ – выборка (тренировочные данные) объема n , $x_i \in X$, $i \in \{1, 2, \dots, n\}$ и B – множество алгоритмов классификации, то есть

$$B = \{b \mid b : X \rightarrow \{-1, 1\}\}.$$

При построении ансамбля будем рассматривать корректирующее правило взвешенного голосования, тогда ансамбль может быть задан аналитическим выражением вида

$$a(x) = \text{sign} \left(\sum_{t=1}^T \omega_t b_t(x) \right),$$

где $b_t(x) \in B$, ω_t – веса (не обязательно нормированные), $t \in \{1, 2, \dots, T\}$, T – количество обученных алгоритмов. Подбор же как алгоритмов классификации, так и весов перед ними, будем производить итеративно, исходя из требования минимизации количества ошибок на тренировочных данных. Количество ошибок может быть задано следующим аналитическим выражением:

$$Q_T = \sum_{i=1}^n \mathbb{I}(a(x_i) \neq y_i) = \sum_{i=1}^n \mathbb{I} \left(y_i \sum_{t=1}^T \omega_t b_t(x_i) < 0 \right),$$

где $x_i \in X$, $\mathbb{I}$ – индикатор. Действительно, каждое слагаемое в первом выражении равно единице только в том случае, когда ансамбль выдает класс, отличный от истинного класса тренировочного объекта. Конечно, наша задача – в минимизации функции Q_T на тренировочных данных, ведь чем меньше число ошибок, тем лучше.

Минимизация написанного выражения – задача затруднительная, так как написанная функция не является гладкой. Однако, если мы найдем какую-то «хорошую» функцию $\tilde{Q}_T$, для которой $Q_T \leq \tilde{Q}_T$, то, минимизируя последнюю, будет уменьшаться и исходная функция. Ясно, что так как

$$Q_T = \sum_{i=1}^n \mathbb{I} \left(y_i \sum_{t=1}^T \omega_t b_t(x_i) < 0 \right) \leq \sum_{i=1}^n \exp \left(-y_i \sum_{t=1}^T \omega_t b_t(x_i) \right) = \tilde{Q}_T,$$

то написанная функция $\tilde{Q}_T$ подходит. Именно она и используется в алгоритме адаптивного бустинга. Перепишем ее в виде

$$\tilde{Q}_T = \sum_{i=1}^n \exp \left(-y_i \sum_{t=1}^{T-1} \omega_t b_t(x_i) \right) e^{-y_i \omega_T b_T(x_i)}$$

и обозначим

$$\alpha_i = \exp \left(-y_i \sum_{t=1}^{T-1} \omega_t b_t(x_i) \right).$$

Нормируем коэффициенты так, чтобы их сумма была равна единице, тогда получим

$$\alpha_0 = \sum_{i=1}^n \alpha_i, \quad \alpha_i = \frac{\alpha_i}{\alpha_0}, \quad i \in \{1, 2, \dots, n\}.$$

Замечание 3.3.1 Последнее равенство следует понимать так: сначала вычисляется отношение α_i/α_0 , а затем новое значение присваивается переменной α_i . Часто это обозначают либо

$$\alpha_i \leftarrow \frac{\alpha_i}{\alpha_0}, \quad \text{либо} \quad \alpha_i := \frac{\alpha_i}{\alpha_0}.$$

Замечание 3.3.2 Полезно заметить, что выражение для $\tilde{Q}_T$ переписывается в виде

$$\tilde{Q}_T = \sum_{i=1}^n \alpha_i e^{-y_i \omega_T b_T(x_i)}$$

и коэффициенты α_i не зависят от алгоритма b_T , а зависят только от алгоритмов b_i с номерами $i < T$, а значит могут быть найдены итерационно.

Конечно, нужно пояснить, откуда берутся коэффициенты ω_i , $i \in \{1, 2, \dots, T\}$, но мы это сделаем непосредственно в алгоритме.

Итак, запишем алгоритм для тренировочного набора данных $x_1, x_2, \dots, x_n$ объема n , заданного множества алгоритмов B и числа используемых алгоритмов T по шагам:

1. Инициализировать начальные веса следующим образом:

$$\alpha_1 = \alpha_2 = \dots = \alpha_n = \frac{1}{n}.$$

2. Для всех $t \in \{1, 2, \dots, T\}$

- (a) Найти алгоритм b_t из множества B , минимизирующий количество ошибок на тренировочном наборе данных, с весами $\alpha_1, \alpha_2, \dots, \alpha_n$:

$$b_t = \arg \min_{b \in B} \sum_{i=1}^n \alpha_i \cdot \mathbb{I}(b(x_i) \neq y_i).$$

Если таких алгоритмов несколько, выбрать любой.

(b) Пусть

$$\varepsilon_t = \sum_{i=1}^n \alpha_i \cdot \mathbb{I}(b_t(x_i) \neq y_i)$$

– найденное минимальное значение на алгоритме b_t . Положить

$$\omega_t = \frac{1}{2} \ln \frac{1 - \varepsilon_t}{\varepsilon_t}.$$

(c) Пересчитать $\alpha_i = \alpha_i e^{-y_i \omega_t b_t(x_i)}$, $i \in \{1, 2, \dots, n\}$.

(d) Нормировать полученные коэффициенты:

$$\alpha_0 = \sum_{i=1}^n \alpha_i, \quad \alpha_i = \frac{\alpha_i}{\alpha_0}, \quad i \in \{1, 2, \dots, n\}.$$

3. Составить итоговый ансамбль в виде

$$a(x) = \text{sign} \left(\sum_{t=1}^T \omega_t b_t(x) \right).$$

Можно доказать, что при некоторых необременительных ограничениях на множество алгоритмов B , минимум $\tilde{Q}_T$ достигается именно при таком выборе параметров ω_i , $i \in \{1, 2, \dots, T\}$

3.3.1 Пример

Давайте рассмотрим следующий «игрушечный пример», чтобы пройти по шагам описанного алгоритма. Мы опустим подбор оптимального алгоритма b_t на шаге 2а, подбирая его «на глаз», лишь для упрощения изложения.

Итак, имеется выборка объема $n = 12$, желтые точки соответствуют классу «+1», зеленые – классу «−1». Пусть базовые алгоритмы классификации способны разделять плоскость либо горизонтальными, либо вертикальными прямыми. Легко увидеть, что ни одна из таких прямых не разделит плоскость так, чтобы не было ошибок классификации. Пусть $T = 3$, тем самым наша задача – корректным образом построить три алгоритма классификации, снабдив каждый корректным весом. Рассмотрим подробно каждую итерацию:

1. На первом инициализируем значения $\alpha_1, \alpha_2, \dots, \alpha_{12}$, они равны

$$\alpha_1 = \frac{1}{12}, \alpha_2 = \frac{1}{12}, \dots, \alpha_{12} = \frac{1}{12}.$$

AdaBoost.png

Рис. 10: Три итерации алгоритма AdaBoost.

Пусть базовый алгоритм $b_1(x)$ представлен горизонтальной линией (верхнюю полуплоскость он относит к зеленым, а нижнюю — к желтым). Мы видим, что два зеленых объекта классифицированы неверно, они выделены в таблице синим. Найдем ε_1 из соотношения

X	(4, 1)	(2, 4)	(5, 1)	(3, 2)	(1, 5)	(3, 5)	(2, 1)	(2, 2)	(2, 5)	(4, 3)	(2, 3)	(5, 2)
y_i	+1	-1	+1	+1	-1	-1	+1	+1	-1	-1	+1	-1
$b_1(x_i)$	+1	-1	+1	+1	-1	-1	+1	+1	-1	+1	+1	+1

$$\varepsilon_1 = \frac{1}{12} \sum_{i=1}^{12} \mathbb{I}(b_1(x_i) \neq y_i) = \frac{1}{12} \cdot 2 \approx 0.167.$$

Теперь найдем значение ω_1 :

$$\omega_1 = \frac{1}{2} \ln \left(\frac{1 - \varepsilon_1}{\varepsilon_1} \right) \approx \frac{1}{2} \ln \left(\frac{1 - 0.167}{0.167} \right) \approx 0.805.$$

Найдем новые веса для объектов, пересчитав их, используя соотношения

$$\alpha_i = \alpha_i e^{-y_i \omega_1 b_1(x_i)}, i \in \{1, 2, \dots, n\}.$$

Например, α_1 находится, как

$$\alpha_1 \approx \frac{1}{12} \cdot e^{-1 \cdot 0.805} \approx 0.037.$$

Остальные значения получаются аналогичным образом и представлены в таблице. Теперь нормируем найденные веса.

Зная значение всех весов, находим их сумму $\alpha_0 \approx 0.745$ и делим каждый из весов на α_0 , получим значения указанные в последней строке таблицы – это новые веса, используемые на следующей итерации (конечно, округленные).

X	(4, 1)	(2, 4)	(5, 1)	(3, 2)	(1, 5)	(3, 5)	(2, 1)	(2, 2)	(2, 5)	(4, 3)	(2, 3)	(5, 2)
y_i	+1	-1	+1	+1	-1	-1	+1	+1	-1	-1	+1	-1
$b_1(x_i)$	+1	-1	+1	+1	-1	-1	+1	+1	-1	+1	+1	+1
$\alpha_i \cdot e^{-y_i \omega_1 b_1(x_i)}$	0.037	0.037	0.037	0.037	0.037	0.037	0.037	0.037	0.037	0.186	0.037	0.186
$\frac{\alpha_i \cdot e^{-y_i \omega_1 b_1(x_i)}}{\alpha_0}$	0.05	0.05	0.05	0.05	0.05	0.05	0.05	0.05	0.05	0.25	0.05	0.25

Как видим, почти все веса стали меньше начальных ($0.05 < 0.083$), кроме двух, которые, наоборот, увеличились. Не сложно догадаться, что это и есть те объекты, которые были классифицированы неверно. На следующей итерации следующий базовый алгоритм придаст им большее внимание.

2. Переходим ко второму шагу. Теперь веса таковы:

$$\alpha_i \approx \begin{cases} 0.25, & i \in \{10, 12\} \\ 0.05, & i \in \{1, 2, \dots, 12\} \setminus \{10, 12\} \end{cases}.$$

Для наглядности изменим размер точек на рисунке [10](#) в соответствии с изменившимися весами. Пусть теперь выбранный базовый алгоритм $b_2(x)$ делит плоскость так, как показано на среднем рисунке (верх снова относится к зеленому классу, а низ – к желтому). Классифицируем каждый объект, используя выбранный алгоритм, данные представлены в таблице.

X	(4, 1)	(2, 4)	(5, 1)	(3, 2)	(1, 5)	(3, 5)	(2, 1)	(2, 2)	(2, 5)	(4, 3)	(2, 3)	(5, 2)
y_i	+1	-1	+1	+1	-1	-1	+1	+1	-1	-1	+1	-1
$b_2(x_i)$	+1	-1	+1	-1	-1	-1	+1	-1	-1	-1	-1	-1
α_i	0.05	0.05	0.05	0.05	0.05	0.05	0.05	0.05	0.05	0.25	0.05	0.25

Вес элементов, на которых допущена ошибка, равен 0.05, а значит

$$\varepsilon_2 \approx 3 \cdot 0.05 = 0.15.$$

и тогда:

$$\omega_2 \approx \frac{1}{2} \ln \left(\frac{1 - 0.15}{0.15} \right) \approx 0.867.$$

Повторяем расчет аналогично проделанному в первом пункте, нормируем веса ($\alpha_0 \approx 0.714$) и представим данные в виде таблицы:

X	(4, 1)	(2, 4)	(5, 1)	(3, 2)	(1, 5)	(3, 5)	(2, 1)	(2, 2)	(2, 5)	(4, 3)	(2, 3)	(5, 2)
y_i	+1	-1	+1	+1	-1	-1	+1	+1	-1	-1	+1	-1
$b_2(X)$	+1	-1	+1	-1	-1	-1	+1	-1	-1	-1	-1	-1
$\alpha_i \cdot e^{-y_i \omega_t b_2(x_i)}$	0.021	0.021	0.021	0.119	0.021	0.021	0.021	0.119	0.021	0.105	0.119	0.105
$\frac{\alpha_i \cdot e^{-y_i \omega_t b_2(x_i)}}{\alpha_0}$	0.029	0.029	0.029	0.167	0.029	0.029	0.029	0.167	0.029	0.147	0.167	0.147

3. Остался последний шаг алгоритма. Как мы видим, теперь у нас имеется три категории весов: 0.029 – для точек, которые все рассмотренные ранее базовые алгоритмы классифицировали верно (вес для них, кстати, продолжает уменьшаться), 0.167 – для трех точек, которые были классифицированы неверно (веса перед ними увеличились), и 0.147 – для точек, при которых на прошлой итерации вес был увеличен, но которые на этот раз были классифицированы верно, поэтому их веса немного, но уменьшились.

Теперь веса:

$$\alpha_i \approx \begin{cases} 0.029, & i \in \{1, 2, 3, 5, 6, 7, 9\} \\ 0.147, & i \in \{10, 12\} \\ 0.167, & i \in \{4, 8, 11\} \end{cases}.$$

Пусть выбранный алгоритм $b_3(x)$ делит плоскость вертикальной прямой, как показано на нижнем рисунке (классификация такая: слева – желтые, справа – зеленые). Видно, что классификатор ошибается в шести случаях (соответствующие столбцы закрашены в таблице и все имеют одинаковый вес).

X	(4, 1)	(2, 4)	(5, 1)	(3, 2)	(1, 5)	(3, 5)	(2, 1)	(2, 2)	(2, 5)	(4, 3)	(2, 3)	(5, 2)
y_i	+1	-1	+1	+1	-1	-1	+1	+1	-1	-1	+1	-1
$b_3(x_i)$	-1	+1	-1	+1	+1	+1	+1	+1	+1	-1	+1	-1
ω_3	0.029	0.029	0.029	0.167	0.029	0.029	0.029	0.167	0.029	0.147	0.167	0.147

Тогда доля ошибок составит

$$\varepsilon_3 \approx 6 \cdot 0.029 \approx 0.174,$$

а значение $\omega_3 \approx 0.779$. Все результаты (пересчет и нормирование весов) представим в таблице:

Итак, финальный алгоритм строится следующим образом:

$$a(x) = \text{sign} \left(\sum_{t=1}^3 \omega_t b_t(x) \right) \approx \text{sign} (0.805 \cdot b_1(x) + 0.867 \cdot b_2(x) + 0.779 \cdot b_3(x)).$$

X	(4, 1)	(2, 4)	(5, 1)	(3, 2)	(1, 5)	(3, 5)	(2, 1)	(2, 2)	(2, 5)	(4, 3)	(2, 3)	(5, 2)
y_i	+1	-1	+1	+1	-1	-1	+1	+1	-1	-1	+1	-1
$b_3(X)$	-1	+1	-1	+1	+1	+1	+1	+1	+1	-1	-1	+1
$\omega_3(i) \cdot e^{-\alpha_3 y_i b_3(X)}$	0.064	0.064	0.064	0.076	0.064	0.064	0.013	0.076	0.064	0.067	0.076	0.067
$\frac{\omega_3(i) \cdot e^{-\alpha_3 y_i b_3(X)}}{Z_3}$	0.084	0.084	0.084	0.1	0.084	0.084	0.018	0.1	0.084	0.089	0.1	0.089

Тогда классификация тестового объекта, скажем, $x = (4, 1)$ с помощью ансамбля может быть получена как:

$$a(x) = \text{sign}(0.805 \cdot (+1) + 0.867 \cdot (+1) + 0.779 \cdot (-1)) = \text{sign}(0.893) = +1.$$

Ансамбль его относит к желтым.



Рис. 11: Итоговый классификатор, полученный алгоритмом AdaBoost.

3.4 Стекинг (Stacking)

В традиционных подходах ансамблевых моделей, что рассматривались ранее, мы использовали несколько базовых моделей на различных комбинациях входных данных и объединяли их результаты с помощью голосования, взвешенного голосования, усреднения результатов и т.д. В то же время резонно возникает следующий вопрос: почему, собственно, для создания композиции используются такие простые операции как усреднение или голосование? Может, подбор решающего правила доверить очередному алгоритму (т.н. «метаалгоритму») машинного обучения.

Идея стекинга в этом и состоит: обучить несколько разных базовых моделей и объединить их путем обучения так называемой метамодели. Метамодель является, по своей сути, еще одной моделью, которая служит для вывода прогнозов на основе нескольких прогнозов, возвращенных базовыми моделями. В итоге, мы используем не заранее спланированное решение, вроде голосования, а даем возможность выбрать корректирующее правило новой модели.

Существует несколько возможных алгоритмов стекинга, мы рассмотрим лишь один из них. Обратите внимание, в этом алгоритме обозначения несколько отличаются от обозначений в предыдущих пунктах.

1. Пусть $X = \{x_1, x_2, \dots, x_n\}$ – обучающая выборка с откликами $Y = \{y_1, y_2, \dots, y_n\}$, $b_1, b_2, \dots, b_k$ – базовые алгоритмы.
2. Разделить X на k непересекающихся почти одинаковых по размеру блоков (folds)

$$X = X_1 \cup X_2 \cup \dots \cup X_k.$$

Обычно k – это не очень большое число (до 10).

3. Для каждого $i \in \{1, 2, \dots, k\}$ и $j \in \{1, 2, \dots, k\}$ обучить базовый алгоритм b_i на наборе данных

$$X_{[-j]} = X \setminus X_j,$$

и протестировать на X_j , получив набор откликов Y_{ji} .

4. Сформировать метаданные (метапредикторы) для обучения метамодели следующим образом

$$X_{meta} = \begin{pmatrix} Y_{11} & Y_{12} & \dots & Y_{1k} \\ Y_{21} & Y_{22} & \dots & Y_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ Y_{k1} & Y_{k2} & \dots & Y_{kk} \end{pmatrix}.$$

В качестве откликов для обучения метамодели взять отклики исходные отклики Y .

5. Обученная метамодель и считается ансамблем, построенным на основе стекинга.

Замечание 3.4.1 Для применения построенной метамодели к тестовым данным, часто поступают следующим образом:

- Базовые алгоритмы обучают на всей тестовой выборке.
- Тестовое данное подается на вход каждому базовому алгоритму, тем самым получая строку метаданных.
- Строка полученных метаданных подается обученному метаалгоритму.

3.4.1 Пример на пальцах

Чтобы продемонстрировать все этапы алгоритма, рассмотрим следующие данные. Пусть двое играют в дартс. Мишень – круг с центром в точке $(0, 0)$ единичного радиуса. Пусть X_1 откладывается по горизонтали, а X_2 – по вертикали, тогда пара (X_1, X_2) – координаты дротика, попавшего в мишень. Пусть отклик 0 соответствует первому игроку (зеленые точки на рисунке 12), а отклик 1 – второму (желтые точки).

Составим модель классификации, которая будет предсказывать, кто из игроков выполнил бросок, основываясь на координатах попадания дротика.

Для стекинга будем использовать разделение на 4 непересекающихся блока случайным образом. В качестве базовых алгоритмов выберем: k -ближайших соседей ($k = 3$) и дерево принятия решений (максимальная глубина 4, критерий разбиения — энтропия). Для сравнения результатов, применим алгоритмы к исходным данным. Из рисунков видно, что алгоритм k -ближайших соседей (рисунок 13a) допустил три ошибки, а дерево принятия решений (рисунок 13b) – две.

Выполнив четыре итерации алгоритма стекинга, получаем отклики выбранных базовых алгоритмов для каждого из разбиений (блока). Полученные результаты могут быть записаны в табличном виде: В принятых обозначениях получены метаданные (метапредикторы) для обучения метамодели

$$X_{meta} = \begin{pmatrix} 0 & 0 \\ 1 & 1 \\ \vdots & \vdots \\ 1 & 1 \\ 1 & 0 \\ \vdots & \vdots \\ 1 & 1 \end{pmatrix}.$$

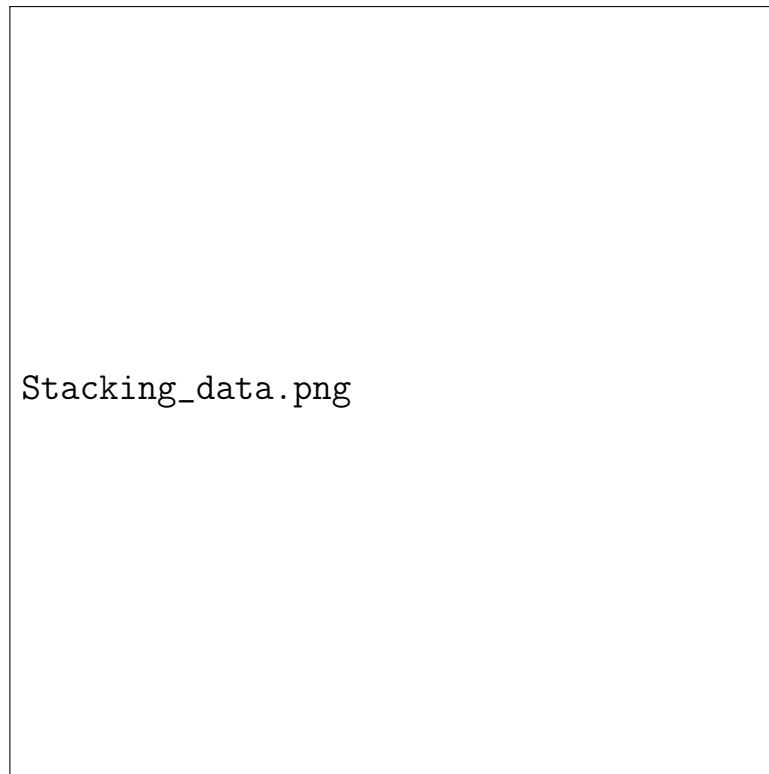


Рис. 12: Исходные данные попаданий в дартс.

Номер блока	X_1	X_2	Y_{kNN}	Y_{DT}	Y
4	0.94	-0.18	0	0	0
3	0.12	-0.93	1	1	0
...	...	...	...	...	...
3	0.18	-0.90	1	1	0
2	-0.82	-0.37	1	0	1
...	...	...	...	...	...
1	0.29	-0.04	1	1	1

Метаалгоритм $a(x)$ обучим на предикторах X_{meta} и отклике Y . В качестве метаалгоритма используем логистическую регрессию. Результат представлен на рисунке [14](#) из которого можно заметить, что итоговая модель выбрала черты двух моделей.

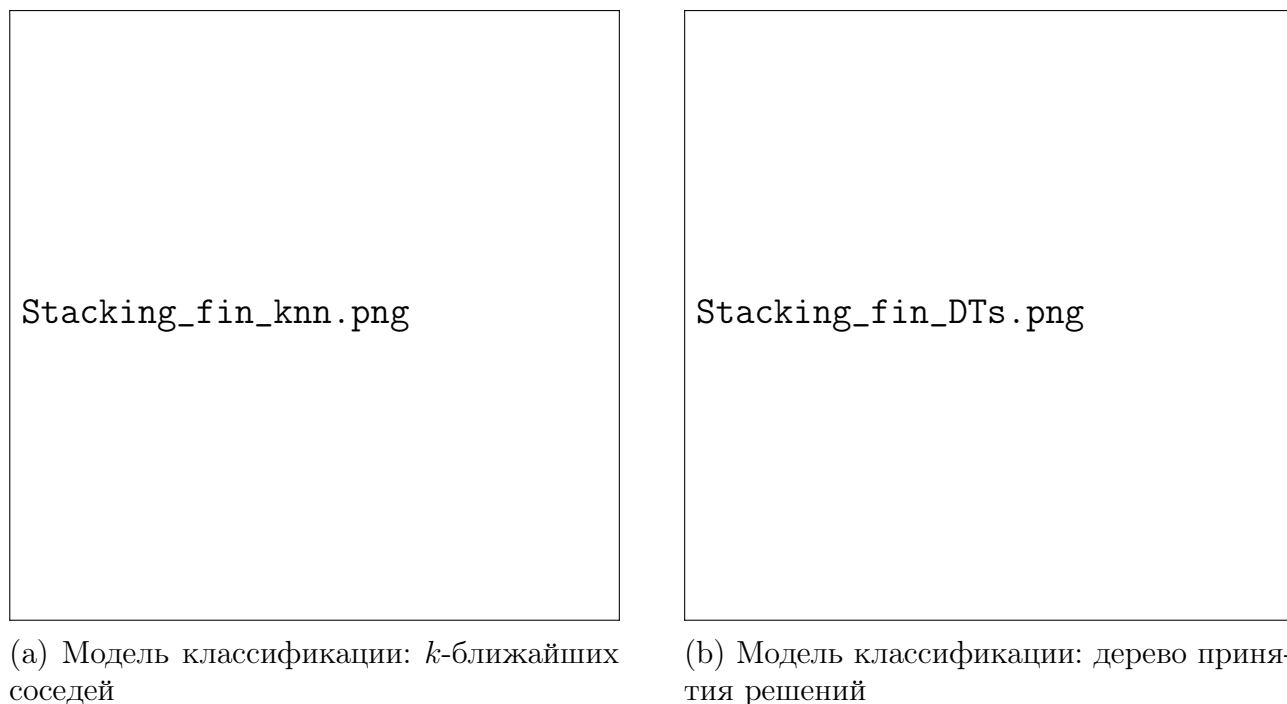


Рис. 13: Сравнение моделей классификации

3.5 Случайный лес

Ранее мы познакомились с алгоритмом деревьев принятия решений. Этот алгоритм очень удобно модифицировать, используя бэггинг: построенное ДПР, как мы видели, может достигать нулевой ошибки на любой обучающей выборке, но, в то же время, может быть очень неустойчивым на тестовой.

Случайный лес – это ансамбль деревьев принятия решений, основанный на бэггинге, но с одним важным дополнением – добавлением еще одной случайности – случайности по признакам. Таким образом, каждое дерево в лесу будет обучено на случайной бутстрэп выборке и некотором наборе признаков из исходного тренировочного набора данных. Все это делает деревья менее похожими между собой, а итоговую модель более устойчивой. Обычно для задачи регрессии рекомендуется использовать $m = \lfloor \frac{p}{3} \rfloor$, а в классификации $m = \lfloor \sqrt{p} \rfloor$ признаков (квадратные скобки обозначают целую часть числа).

Резюмируя, запишем алгоритм построения случайного леса. Пусть имеется обучающая выборка X объема n с p предикторами, тогда для каждого дерева $t = \{1, 2, \dots, T\}$ необходимо:

1. Сформировать бутстрэп выборку X_t .
2. Из исходных p предикторов, случайным образом выбрать m предикторов.



Рис. 14: Модель классификации: стекинг.

3. Сформировать дерево принятия решений на основе выборки X_t , оставив m предикторов.
4. Повторить шаги 1-3 заданное число T раз.
5. В зависимости от задачи, сформировать решающее правило и ансамбль.

Еще одно достоинство случайного леса заключается в том, что для оценки вероятности ошибочной классификации нет необходимости использовать кросс-проверку или тестовую выборку. Ведь при формировании деревьев используются бутстрэп, а как мы знаем бутстрэп выборка состоит из около 63% объектов исходной выборки. А значит, на оставшихся, можно выполнить оценку.

Обратимся к синтетическим данным и рассмотрим задачу классификации, сравнив три подхода: деревья принятия решений, бэггинг над ними и случайный лес (рисунок 15). Как мы видим (впрочем, мы это и так знали), деревья решений дают нам переобучение, что отражается на рисунке в виде ровных угловатых границ, уходящих далеко от объектов. В случае бэггинга и случайного леса границы уже более гладкие, нет явных выступов: по сути, область выглядит более цельной. В целом проблема переобучения практически не свойственна случайному лесу, в особенности, если в лесу достаточно деревьев.



RandomForest.png

Рис. 15: Сравнение моделей.

Однако основным ограничением случайного леса является то, что большое количество деревьев может сделать алгоритм медленным и неэффективным для предсказаний в реальном времени. Как правило, эти алгоритмы быстро обучаются, но довольно медленно создают прогнозы после обучения.